PEKEMEN

Videojuego de combate por consola en Java

Proyecto	Proyecto individual incremental · Primera evaluación · Entrega 1
Alumno	Jorge García González
Módulo / grupo	1.º DAW
Curso	2026/2027
Repositorio GitHub	https://github.com/deniozaque/trabajo-de-programacion
Versión	v1.0-primera-evaluacion

Índice

1. Descripción del videojuego 3
1.1. Temática y objetivo 3
1.2. Reglas básicas 3
1.3. Acciones del jugador en combate 3
1.4. Desarrollo de una partida 4
1.5. Condiciones de victoria y derrota 4
1.6. Estados alterados y acción «Explotar» 4
2. Funcionalidades implementadas 5
2.1. Diferencias respecto a la propuesta inicial 6
2.2. Catálogo de pekemens 6
3. Diseño e implementación 7
3.1. Estructura del programa 7
3.2. Variables y constantes 7
3.3. Operadores y conversiones de tipo 8
3.4. Estructuras de control 9
3.5. Entrada y salida por consola 9
3.6. Flujo de un turno de combate 10
3.7. Uso de arrays (explicación detallada) 10
4. Pruebas realizadas 12
4.1. Pruebas de combate y menús 12
4.2. Cómo se documentan las pruebas con aleatoriedad 13
4.3. Plan de pruebas del modo historia 13
5. Dificultades, soluciones y ampliaciones 14
5.1. Dificultades encontradas y soluciones 14
5.2. Limitaciones conocidas 14
5.3. Ampliaciones propuestas para la segunda evaluación 15
6. Revisión tras el feedback del 6-11-26 15
6.1. Correcciones realizadas 15
6.2. Decisiones de diseño y puntos que no afectan al programa 16
6.3. Pendiente para la segunda evaluación 16

Este proyecto ha sido totalmente pensado, elaborado y programado por el autor Jorge, el único apoyo en la inteligencia artificial, internet o personas conocidas ha sido en la redacción de este FDP y en el Readme.

1. Descripción del videojuego

1.1. Temática y objetivo

Pekemen es un videojuego de combate por turnos que se juega por consola. El jugador elige una criatura inicial (los pekemens) de uno de los tres elementos del juego (agua, planta o fuego) y la enfrenta a la máquina. Cada criatura tiene vida, un daño máximo y un tipo; además, los tipos se relacionan entre sí con ventajas y desventajas.

El programa ofrece dos modos de juego:

- **Combate singular:** una única batalla entre la criatura elegida y un rival aleatorio. Termina cuando uno de los dos pierde.
- **Modo historia:** tras ganar el primer combate, el jugador decide entre luchar contra un líder de gimnasio o buscar pekemens salvajes para matarlos (y ganar experiencia) o capturarlos (y ampliar su equipo). El objetivo es derrotar a los tres líderes (agua, fuego y planta) y al rival final, que usa un equipo de seis criaturas.

1.2. Reglas básicas

- **Ventaja de tipos:** fuego gana a planta, planta gana a agua y agua gana a fuego. El atacante con ventaja multiplica su daño por 1,25; con desventaja, por 0,75. Entre tipos iguales, o si interviene el tipo normal, el multiplicador es 1,0. En esta versión el multiplicador se calcula siempre desde el pekemen del jugador (ver 6.3, punto 17).
- **Daño:** cada ataque calcula una potencia aleatoria entre 10 y (daño máximo + 1) × 10 y la multiplica por la ventaja. Hay un 10 % de probabilidad de golpe crítico (multiplicador 1,1).
- **Equipo:** el jugador puede tener hasta 6 pekemens. Si el pekemen activo cae, debe sacar otro con vida; si no le queda ninguno, pierde el combate.
- **Poción:** cada bando dispone de una sola poción por combate que recupera 20 puntos de vida. No hay tope de vida máxima: la poción puede dejar al pekemen por encima de su vida inicial (decisión de diseño, ver 6.2).
- **Experiencia y niveles:** al derrotar a un salvaje, todo el equipo recibe su experiencia. Cuando la experiencia de un pekemen llega a 100 o más, sube un nivel y se le restan 100 puntos; el nivel máximo es 4. En esta versión el nivel es un contador de progreso: no modifica la vida, el daño máximo ni ninguna otra característica. Además, la experiencia solo se guarda en la matriz cuando el pekemen sube de nivel (limitación conocida, ver 6.3, punto 15).

1.3. Acciones del jugador en combate

En cada turno el jugador elige una de las siguientes acciones. Todas modifican el estado de la partida:

N.º	Acción	Efecto sobre el estado de la partida
1	Atacar	Resta vida al rival (potencia aleatoria × ventaja de tipo); 10 % de crítico.
2	Curarte	Suma 20 de vida al jugador (sin tope de vida máxima). Solo una vez por combate; si no quedan pociones lo avisa.
3	Explotar	El rival queda a 0 de vida (victoria inmediata sobre ese pekemen), pero el jugador pierde la mitad de su vida. Justificación y límites en el apartado 1.6.
4	Atacar fuerte	40 % de acierto; si acierta, hace el doble de daño. Si falla, informa de ello.

N.º	Acción	Efecto sobre el estado de la partida
5	Protegerte	El rival no realiza su ataque normal en ese turno. El golpe crítico del rival no queda bloqueado (ver 6.3).
6	Efecto especial	Según el tipo: fuego quema (−15 de vida por turno), planta envenena (−25 por turno), agua ahoga (el rival cae tras 3 turnos). Solo funciona contra un tipo distinto. Duración y acumulación en el apartado 1.6.
7	Cambiar pekemen	Recorre el equipo, ofrece los pekemens con vida y sustituye al activo conservando su vida actual.

Además de estas acciones, el menú del mundo (modo historia) permite luchar contra el líder, buscar pekemens salvajes y, dentro de este último, matar o capturar. El menú principal permite elegir modo de juego o salir.

1.4. Desarrollo de una partida

- Se muestra el menú principal (modo historia, combate singular o salir) y se repite hasta que la opción sea válida.
- El jugador elige su pekemen inicial (Peplet, Terteg o Chemchemp). La máquina elige otro de los tres al azar.
- Comienza el combate por turnos. En cada turno se muestra el tipo de ambos, la vida de ambos con una barra de almohadillas (una por cada 10 puntos de vida) y el menú de acciones. Tras la acción del jugador actúan los estados alterados y, después, la máquina.
- Cuando cae un pekemen, el combate continúa con el siguiente disponible de ese bando. Cuando un bando se queda sin criaturas, el combate termina.
- En combate singular se vuelve al menú principal. En modo historia se pasa al menú del mundo y la aventura sigue hasta derrotar al rival final o perder un combate.

1.5. Condiciones de victoria y derrota

- **Victoria en un combate:** todos los pekemens del rival llegan a 0 de vida. El programa muestra «Has ganado».
- **Derrota:** todos los pekemens del jugador llegan a 0 de vida. El programa muestra «Has perdido el combate» y vuelve al menú principal.
- **Victoria del modo historia:** haber derrotado a los tres líderes y al rival final, con lo que termina la aventura.

1.6. Estados alterados y acción «Explotar»

Estados alterados. Los tres estados los provoca la acción 6 (efecto especial) del jugador sobre el pekemen rival, y solo si el tipo del rival es distinto al del jugador. El rival controlado por la máquina no aplica estados. Su funcionamiento en esta versión es el siguiente:

- **Quemadura (fuego):** resta 15 puntos de vida al rival al final de cada turno del jugador. Mientras el rival está quemado, su ataque normal hace 10 puntos menos de daño.
- **Veneno (planta):** resta 25 puntos de vida al rival al final de cada turno.
- **Ahogamiento (agua):** inicia un contador de turnos. El contador avanza en cada turno y el rival cae al final del tercer turno posterior a aquel en que se aplica.
- **Duración:** la quemadura y el veneno no tienen un número fijo de turnos; duran hasta que el pekemen rival cae o hasta que el jugador cambia de pekemen.
- **Acumulación:** los estados se procesan con una cadena if / else if con este orden de prioridad: quemadura, veneno y ahogamiento. Si hay varios activos, en ese turno solo se aplica el de

mayor prioridad (y el contador de ahogamiento no avanza mientras haya otro activo). Se decidió así para mantener sencillo el combate.

- **Cambio de criatura:** cuando el jugador cambia de pekemen, o cuando cae el pekemen rival y entra otro, los tres estados se reinician.

Acción «Explotar». Deja a 0 la vida del pekemen rival activo (cae de inmediato) y el jugador pierde la mitad de su vida actual. Se incluyó como acción de alto riesgo y alta recompensa para situaciones límite, por ejemplo cuando al pekemen del jugador le queda poca vida. Es una acción muy potente: puede derrotar a cualquier pekemen rival, con independencia de su nivel o resistencia, y no tiene límite de usos. Lo que la equilibra es su coste (la mitad de la vida actual, que solo se recupera con la única poción) y que elimina únicamente al pekemen activo, no a todo el equipo rival: frente a un líder (3 pekemens) o al rival final (6) hay que repetirla o combinarla con otras acciones. El rival dispone de una versión de esta acción en su «ataque desesperado»: cuando le queda 10 de vida o menos, con alta probabilidad deja a 0 la vida del pekemen del jugador y pierde la mitad de la suya, salvo que el jugador se haya protegido. Como posible mejora se valorará limitar su uso (por ejemplo, una vez por combate).

2. Funcionalidades implementadas

Todo lo que se enumera a continuación está implementado en el archivo App.java. «Implementada» no significa que funcione correctamente en todos los casos previstos: la columna «Estado» indica las limitaciones conocidas, que se detallan en el apartado 6.3. Las pruebas realizadas se recogen en el apartado 4.

Funcionalidad	Estado
Menú principal repetible con validación de rango (1–3) y salida controlada.	Implementada. Limitación: un texto no numérico cierra el programa (6.3, punto 14).
Elección del pekemen inicial con validación; rival inicial aleatorio.	Implementada. Misma limitación ante texto no numérico (6.3, punto 14).
Combate por turnos con siete acciones y barra de vida en pantalla.	Implementada. Limitaciones del combate en 6.3 (puntos 16 a 18).
Sistema de ventajas por tipo (agua / fuego / planta / normal) y golpes críticos.	Implementada. Limitación: el crítico del rival usa la ventaja del jugador y su ataque normal no lleva ventaja (6.3, punto 17).
Estados alterados: quemadura, veneno y ahogamiento.	Implementada. Solo se aplica un estado por turno (apartado 1.6).
Inteligencia artificial sencilla del rival (curarse, crítico, ataque desesperado, ataque normal).	Implementada.
Equipo de hasta 6 pekemens, cambio voluntario y cambio forzoso al caer el activo.	Implementada. Limitación: la vida se guarda como entero al cambiar de pekemen (6.3, punto 16).

Funcionalidad	Estado
Modo historia: tres líderes con 3 pekemens cada uno y rival final con 6.	Implementada. Se ha corregido el reinicio entre partidas (6.1, punto 5); pendiente de probarla completa (apartado 4.3).
Pekemens salvajes: matar (experiencia y nivel) o capturar (probabilidad de éxito).	Implementada. Limitación: la experiencia solo se guarda al subir de nivel (6.3, punto 15).
Condiciones de victoria y derrota, y regreso al menú principal.	Implementada.

2.1. Diferencias respecto a la propuesta inicial

El proyecto ha cambiado bastante desde la idea inicial. Esta tabla recoge qué se ha modificado respecto a esa idea (los tipos pasan a pekemens y el torneo pasa a modo historia), qué se ha añadido durante el desarrollo y por qué.

Elemento	Cambio	Versión entregada	Motivo
Criaturas	Modificado	Se usaban tipos y pasan a ser pekemens con nombre, tipo, vida, daño máximo, nivel y experiencia. El catálogo final tiene 19 (apartado 2.2).	Con datos propios para cada criatura se pueden formar equipos y guardarlos en matrices (apartado 3.7).
Torneo	Modificado	El torneo se convierte en el modo historia: primer combate, menú del mundo, tres líderes de gimnasio y un rival final con 6 pekemens.	Da más progresión que un torneo y permite combinar combates contra líderes y contra salvajes.
Acciones de combate	Añadido	Se añaden más opciones hasta llegar a siete: atacar, curarte, explotar, atacar fuerte, protegerte, efecto especial y cambiar pekemen.	Más variedad en cada turno, con acciones que modifican el estado de la partida (apartado 1.3).
Equipos y cambio	Añadido	Equipo de hasta 6 pekemens, cambio voluntario y cambio forzoso cuando cae el activo.	Necesario para que el combate pueda continuar con varias criaturas y para usar arrays.
Salvajes, captura y experiencia	Añadido	Se pueden buscar pekemens salvajes para matarlos (experiencia y subida de nivel a los 100 puntos) o capturarlos.	Dan una forma de ampliar el equipo y de progresar en el modo historia.
Estados alterados	Añadido	Quemadura, veneno y ahogamiento, asociados al efecto especial de cada tipo.	Dan más profundidad al combate y usan el tipo de cada pekemen.
Rival controlado por la máquina	Añadido	El rival decide con una IA sencilla: se cura, lanza un crítico, hace un ataque desesperado o ataca normalmente.	Hace que los combates sean menos predecibles.

2.2. Catálogo de pekemens

Datos de la matriz pekemens incluida en el juego:

N.º	Nombre	Tipo	Vida	Daño máx.	Exp. que da
1	Peplet	agua	40	2	40
2	Prenplet	agua	70	4	60
3	Enpeleen	agua	100	6	75
4	Terteg	planta	40	2	40

N.º	Nombre	Tipo	Vida	Daño máx.	Exp. que da
5	Tercel	planta	70	4	60
6	Terterra	planta	100	6	75
7	Chemchemp	fuego	40	2	40
8	Menferme	fuego	70	4	60
9	Enfermet	fuego	100	6	75
10	Bedef	normal	30	2	20
11	Shex	normal	30	2	30
12	Lexe	normal	65	4	50
13	Lexrrey	normal	85	6	60
14	Tepexes	agua	35	3	35
15	Mereene	agua	70	5	50
16	Slegme	fuego	40	4	40
17	Megcerge	fuego	75	5	65
18	Cecnee	planta	45	5	45
19	Cecterne	planta	100	6	55

3. Diseño e implementación

3.1. Estructura del programa

El proyecto consta de una única clase App con el método main. La cabecera es un comentario con autor, fecha y descripción; después van los import (BufferedReader, InputStreamReader y Random), y dentro de main el código se organiza en bloques: constantes, declaración de variables y matrices, inicialización, bucle principal del menú, elección de pekemen inicial, bucle de combate, bucle de turnos y menú del mundo.

3.2. Variables y constantes

Las constantes se declaran con final y en mayúsculas; los nombres de variables siguen la convención camelCase:

```
final double CRITICO = 1.1; // multiplicador del golpe crítico
final String FUEGO = "fuego"; // nombres de los tipos
final String AGUA = "agua";
final String PLANTA = "planta";
final String NORMAL = "normal";
final int NUMERO = 0, NOMBRE = 1, TIPO = 2, VIDA = 3; // índices de columna
final int NIVEL = 4, DANHO = 5, EXPERIENCIA = 6; // (una por línea en App.java)
```

Las variables principales, agrupadas por función y tipo elegido:

Variable	Tipo	Para qué sirve
vidaJugador, vidaRival	double	Vida actual de cada bando; admite decimales porque el daño se multiplica por 1,25, 0,75 o 1,1.
potencia, ventaja	double	Daño base calculado en cada ataque y multiplicador de tipo.
fuerzaMaximaJugador / fuerzaMaximaRival	int	Daño máximo del pekemen activo; limita la potencia aleatoria.
eleccion, eleccionPekemen, eleccionSalvaje	int	Opciones tecleadas en los distintos menús.
posicion, posicionRival	int	Índice del pekemen activo dentro de cada equipo (array).

Variable	Tipo	Para qué sirve
contadorAhogar	int	Turnos transcurridos del efecto «ahogar» (9 significa inactivo).
tipoJugador, tipoRival, inicialRival	String	Tipo de cada pekemen activo y nombre del rival en pantalla.
historia, lider, huir, salir, perder	boolean	Variables de control: modo de juego, fin de menú, fin de combate y salida de la partida (ver explicación debajo).
pocionJugador, pocionRival	boolean	Si cada bando conserva su poción.
quemar, envenenar	boolean	Estados alterados activos sobre el rival.
save (BufferedReader), ram (Random)	objetos	Lectura de teclado y generación de números aleatorios.

Variables de control. Cada una cambia de valor en momentos concretos y entre todas gobiernan los bucles anidados:

- **huir:** vale false al empezar y pasa a true solo al elegir «Salir» en el menú principal. Mientras es false se repite el bucle principal; al ser true se hace un break y el programa termina.
- **salir:** variable auxiliar de los menús de elección de pekemen inicial y de acciones de combate. Se pone a false al abrir el menú y a true cuando el jugador elige una opción válida; el do-while se repite mientras valga false.
- **historia:** true si se elige «Modo historia» y false en «Combate singular». Mientras vale true se repite el bucle que encadena combate y menú del mundo. Vuelve a false cuando el jugador pierde un combate.
- **lider:** vale false al empezar y pasa a true cuando el jugador elige «luchar contra el líder» en el menú del mundo; con lider == true se carga el equipo del siguiente líder (o del rival final) al comenzar el combate. Se pone a false al ganar ese combate.
- **enfrentamientoLiderAgua, enfrentamientoLiderFuego, enfrentamientoLiderPlanta y enfrentamientoRival:** indican cuál es el siguiente combate de la secuencia líder de agua, líder de fuego, líder de planta y rival final. Cada una se desactiva al cargar su combate y activa la siguiente. Cuando se carga el rival final (enfrentamientoRival pasa a false) y se gana, termina la aventura.
- **perder:** se pone a false al empezar cada combate y a true cuando uno de los dos bandos se queda sin pekemens. Aunque su nombre sugiere derrota, también se activa cuando gana el jugador (es un nombre heredado); después se mira la vida de cada bando para saber quién ha ganado. Termina el bucle de turnos.
- **Relación entre ellas:** huir controla el bucle más externo; dentro, historia controla la repetición de combate y menú del mundo; dentro de cada combate, perder controla los turnos y salir los menús. lider y los enfrentamientos solo deciden qué equipo rival se carga. Al empezar cada partida se reinician los marcadores de progreso del modo historia (enfrentamientoLiderAgua, enfrentamientoRival y lider) para que una partida nueva no herede el estado de la anterior.

3.3. Operadores y conversiones de tipo

Se usan operadores aritméticos para calcular el daño (`vidaRival -= potencia * ventaja * CRITICO`), la barra de vida (`(int) vidaJugador / 10`) y la experiencia; relacionales para comparar vida, tipos y opciones (`<=`, `>`, `==`, `!=`); y lógicos (`&&`, `||`, `!`) para combinar condiciones, por ejemplo la ventaja de tipo o las decisiones del rival.

Como los datos de cada pekemen se guardan en un array de String, aparecen muchas conversiones explícitas: `Double.parseDouble` y `Integer.parseInt` para leer vida, daño o experiencia como números, `String.valueOf` y la concatenación con `""` para volver a guardarlos como texto, y el cast `(int)` para truncar la vida al dibujar la barra. Como conversión implícita está la promoción de int a double al operar

con ventaja. La entrada del teclado llega como texto y se convierte con `Integer.parseInt(save.readLine())`.

Inconvenientes de guardar números como texto. Cada lectura obliga a convertir y cada escritura a volver a convertir. El compilador deja de comprobar los tipos: nada impide guardar un texto no numérico en la vida, y el error solo aparecería al ejecutar (`NumberFormatException`). Y cada conversión puede hacer perder información: por ejemplo, al guardar la vida con `(int)` se pierde la parte decimal (limitación conocida, ver 6.3). Se eligió esta estructura porque, con los contenidos de la primera evaluación, permite guardar una criatura completa en un solo vector; con clases (segunda evaluación) cada dato tendría su propio tipo.

3.4. Estructuras de control

Estructura	Dónde se usa y para qué
while / do-while	Bucle principal (while (huir == false)) que mantiene el juego activo; do-while para repetir cada menú hasta recibir una opción válida; bucle de turnos (while (perder == false)) y bucle del modo historia (while (historia == true)).
for	Recorre los equipos (arrays): comprobar si quedan pekemens vivos, buscar un hueco libre, listar candidatos al cambio, repartir experiencia.
if / else if / else	Reglas del juego: ventaja de tipos, crítico, decisiones de la máquina, victoria y derrota.
switch	Selección de opciones de los menús (principal, pekemen inicial, acciones de combate, salvaje: matar o capturar).
break	Sentencia de salto: sale del switch tras cada caso, del bucle de menú al elegir salir y de los recorridos con for cuando ya se ha encontrado lo buscado (por ejemplo, el primer hueco libre del equipo).

Salida de los bucles. Qué bucle abandona cada break y cómo termina cada bucle:

- **Menú principal (do-while):** los break de cada case solo salen del switch; el do-while termina porque su condición deja de cumplirse (opción 1, 2 o 3). Si se eligió «Salir» (huir == true), un break posterior sale del while principal y el programa termina.
- **Bucle de turnos (do-while con perder):** no usa break; termina cuando un bando se queda sin pekemens (perder = true). En el menú de acciones, cada break sale solo del switch, y el do-while de salir repite el menú si la opción no era válida, sin que el rival actúe.
- **Victoria, derrota y cambio de criatura:** si cae un pekemen y quedan otros, no se sale de ningún bucle: se pide elegir otro (bucle while (!cambiado), cuyo break interno sale solo del recorrido) y el combate continúa. Al terminar el combate, un break sale del bucle do-while (historia == true) cuando el jugador pierde (historia = false), cuando gana en combate singular o cuando se ha derrotado al rival final; en los tres casos se vuelve al bucle principal, que muestra de nuevo el menú principal.
- **Recorridos de equipos (while con contador o for):** el break sale únicamente del recorrido en cuanto se encuentra lo buscado (un pekemen vivo, el primer hueco libre, el siguiente rival con vida).

3.5. Entrada y salida por consola

La entrada se lee con BufferedReader. Cada menú muestra las opciones numeradas y pide «Eleccion:». La salida usa System.out.printf con formato: la vida se muestra con dos decimales (%.2f) y acompañada de una barra de almohadillas generada con `"#".repeat(...)`, de forma que el jugador ve de un vistazo cómo evoluciona el combate:

```
Tu vida es ### 36.59 y la del rival es ###### 60.00
Que quieres hacer
1. Atacar
2. Curarte
...
7. Cambiar pekemen
```

```
Elecion:
```

3.6. Flujo de un turno de combate

Cada vuelta del bucle de turnos sigue siempre el mismo orden, que es la base de las reglas del juego:

1. Se muestra el estado: barras y valores de vida de ambos bandos y el menú de acciones.
2. Se lee la opción del jugador y se ejecuta con un switch (daño, curación, efecto, cambio, etc.).
3. Se aplican los estados alterados sobre el rival: quemadura (−15), veneno (−25) o avance del contador de ahogamiento.
4. Si el rival sigue con vida, la máquina actúa: se cura, lanza un crítico, hace un ataque desesperado o ataca normalmente, según su vida y el azar.
5. Se comprueba si ha caído algún pekemen: se busca el siguiente con vida de ese bando o se marca el fin del combate.
6. Si el combate no ha terminado, se vuelve al primer paso.

La ventaja de tipos y el daño se calculan con condicionales y operadores, como en este fragmento (versión simplificada del código real: el comentario sustituye a las combinaciones inversas):

```
if ((tipoJugador.equals(FUEGO) && tipoRival.equals(PLANTA))
        || (tipoJugador.equals(PLANTA) && tipoRival.equals(AGUA))
        || (tipoJugador.equals(AGUA) && tipoRival.equals(FUEGO))) {
    ventaja = 1.25; // el jugador tiene ventaja
} else if (/* combinaciones inversas */) {
    ventaja = 0.75; // el jugador tiene desventaja
} else if (tipoJugador.equals(tipoRival) || tipoJugador.equals(NORMAL) ||
tipoRival.equals(NORMAL)) {
    ventaja = 1.0;
}
// ataque básico: potencia aleatoria entre 10 y (fuerza + 1) * 10
potencia = (ram.nextDouble(fuerzaMaximaJugador) + 1) * 10;
vidaRival -= potencia * ventaja;
```

Las cadenas se comparan con `.equals()`, que compara el contenido del texto; el operador `==` compara referencias (si dos variables apuntan al mismo objeto) y solo funcionaría por casualidad con literales compartidos. Se usan las constantes (FUEGO, AGUA, PLANTA, NORMAL) en lugar de literales para tener un único criterio. Esta ventaja se calcula desde el pekemen del jugador: se aplica a sus ataques, pero el ataque normal del rival no lleva multiplicador y su golpe crítico reutiliza el del jugador (limitación conocida, ver 6.3). El cálculo está repetido en varios puntos del código (al elegir pekemen, al cambiarlo, al caer el activo y cuando entra un nuevo rival) y se agrupará en un método en la segunda evaluación.

3.7. Uso de arrays (explicación detallada)

El juego depende de los arrays: sin ellos no se podría guardar el catálogo de criaturas, los equipos ni los datos del pekemen salvaje. Se usan los siguientes:

Array	Declaración	Qué almacena
pekemens	String[19][8]	Catálogo fijo de las 19 criaturas del juego.
tusPekemens	String[6][8]	Equipo del jugador (hasta 6 pekemens).
pekemensRival	String[6][8]	Equipo del rival en cada combate (hasta 6 pekemens).
liderAgua / liderFuego / liderPlanta	String[3][8]	Equipo de cada líder del modo historia.
pekemenSalvaje	String[2]	Vector unidimensional con el nombre y la experiencia del salvaje encontrado.

Vector de cada pekemen. Cada fila de esas matrices es un vector unidimensional de 8 posiciones (String[8]) con los datos de una criatura en un orden fijo:

Índice	0	1	2	3	4	5	6	7
Dato	número	nombre	tipo	vida	nivel	daño máximo	experiencia actual	exp. que da al ser derrotado

Esto permite acceder al dato concreto con dos índices: `tusPekemens[posicion][VIDA]` es la vida del pekemen activo (VIDA es una constante que vale 3, para no tener que recordar qué columna es cada dato). En algunos puntos del código aún se escribe el índice numérico directamente (por ejemplo `[1]` o `[3]`) y se sustituirá por la constante. Para evitar que dos equipos compartan la misma criatura, al copiar una fila del catálogo se usa clone(), de modo que cada equipo tiene su propio vector y modificar la vida de uno no altera el catálogo:

```
tusPekemens[0] = pekemens[0].clone(); // el jugador recibe una copia de Peplet
```

Qué hace clone() y qué pasaría sin él. Una fila de la matriz es una referencia a un vector. `tusPekemens[0] = pekemens[0];` solo copiaría la referencia: las dos posiciones apuntarían al mismo vector en memoria, de modo que cambiar `tusPekemens[0][VIDA]` cambiaría también el catálogo, y si el rival recibiera la misma fila los dos equipos compartirían los mismos datos. `clone()` crea un vector nuevo de 8 posiciones con los mismos valores; como los String no se pueden modificar, esa copia es suficiente.

Operaciones que se realizan sobre los arrays:

- **Consulta y actualización:** al empezar cada turno se leen del array el tipo, la vida y el daño máximo del pekemen activo; la vida se escribe de vuelta en la matriz al cambiar de pekemen o al caer en combate (`tusPekemens[posicion][VIDA] = ...`), y al cambiar se guarda truncada a entero (limitación conocida, ver 6.3).
- **Recorrido y conteo/búsqueda:** para saber si el equipo sigue vivo se recorre con un for hasta encontrar algún pekemen con vida mayor que 0; el mismo patrón localiza al siguiente rival disponible.
- **Selección:** el cambio de pekemen recorre el equipo, descarta el activo y las posiciones vacías y ofrece el resto al jugador.
- **Actualización masiva:** al matar a un salvaje se recorre todo el equipo para sumar experiencia y subir de nivel si procede.
- **Inserción:** al capturar, se busca la primera posición libre (`tusPekemens[contador][NOMBRE] == null`) del equipo y se guarda allí una copia de la criatura capturada.

```
for (int i = 0; i < tusPekemens.length; i++) {
    if (tusPekemens[i] != null && tusPekemens[i][NOMBRE] != null && i != posicion
            && Double.parseDouble(tusPekemens[i][VIDA]) > 0) {
        // este pekemen está disponible para el cambio
    }
}
```

Control de índices. Los recorridos usan `i < array.length`, nunca un número fijo. Antes de leer una posición del equipo se comprueba que no sea null (hueco vacío), y la condición se escribe de forma que, si falla, Java no evalúa el acceso al dato (evaluación en cortocircuito de &&). Los índices del catálogo que se usan para salvajes (9 a 12) y líderes están dentro de las 19 posiciones de pekemens.

Qué es null y por qué aparece. `new String[6][8]` reserva seis filas de ocho posiciones, y todas valen null (sin valor, ninguna referencia a un String) hasta que se les asigna algo. Por eso un hueco libre del equipo es una fila con todos sus elementos a null, y se detecta con `tusPekemens[i][NOMBRE] == null`. Si se usa ese null como si fuera un dato, el programa falla: `Double.parseDouble(tusPekemens[i][VIDA])` sobre un hueco lanzaría una `NullPointerException`; y si

una fila entera fuera null, leer `tusPekemens[i][NOMBRE]` también la lanzaría. Por eso se comprueba antes de leer.

Por qué un array. El equipo tiene un número máximo de integrantes conocido (6) y hay que acceder a ellos por posición, recorrerlos y modificarlos; un conjunto de variables sueltas habría sido inmanejable. Guardar todos los datos como texto en un solo vector simplifica copiar una criatura completa, aunque obliga a convertir tipos (ver 3.3) y es una decisión que se podría mejorar (apartado 5).

4. Pruebas realizadas

4.1. Pruebas de combate y menús

Las pruebas se han hecho ejecutando App.java con una versión de Java 21 y simulando la entrada de teclado. Como el juego usa números aleatorios, en las partidas completas el resultado exacto cambia de una ejecución a otra; lo que se comprueba es el comportamiento esperado, no cifras concretas.

N.º	Prueba	Entrada	Resultado esperado	Resultado obtenido	Estado
1	Salir desde el menú (normal)	3	Muestra «Adios» y el programa termina	Muestra «Adios» y termina	Correcta
2	Opción inválida en el menú principal	7, luego 3	Muestra «del 1 al 3» y vuelve a preguntar	Mensaje de error y se repite el menú	Correcta
3	Pekemen inicial inválido	2, 9, 1	Avisa de que se elija entre 1 y 3 y repite	Muestra «Escoge entre el 1 y el 3» y repite	Correcta
4	Opción inválida en combate (límite)	9	Mensaje de opción válida del 1 al 7	Muestra el mensaje y vuelve a pedir la opción; el rival no actúa (versión actual). En la versión inicial el rival atacaba igualmente.	Corregida (repetir la prueba)
5	Curarse con poción	2	Vida +20 y poción gastada	«Has gastado tu ultima pocion»; vida 16,59 pasó a 36,59	Correcta
6	Curarse sin poción (límite)	2 (segunda vez)	Avisa de que no quedan pociones	«No te has podido curar porque no tienes pociones»	Correcta
7	Protegerse	5	El rival no hace daño normal	«Te has protegido»; la vida se mantiene en 18,97	Correcta
8	Efecto especial (agua contra planta)	6	Aplica el efecto de ahogar	«Le has ahogado»; el contador del efecto empieza	Correcta
9	Explotar	3	Rival a 0 de vida; victoria	«El rival ha sido derrotado», «Has ganado» y vuelta al menú	Correcta
10	Ganar atacando	1 repetido	La vida del rival baja hasta 0 y gana	Vida del rival 40, 33,25 y derrotado; «Has ganado»	Correcta
11	Perder el combate	2 repetido	Caen todos los pekemens del jugador; derrota	«Tu Peplet ha sido derrotado», «Has perdido el combate» y menú principal	Correcta
12	Cambiar de pekemen sin otros disponibles (límite)	7	Avisa de que no hay otros con vida	«No tienes otros Pekemens disponibles con vida para cambiar.»	Correcta
13	Modo historia: tras ganar, menú del mundo	1, 1, 3	Tras la victoria aparece el menú líder / salvajes	Aparece el menú y el modo no termina	Correcta
14	Salvaje: capturar con hueco libre	2, 2, no	Intenta la captura en el primer hueco libre (60 %)	En 3 de 4 ejecuciones «Le has capturado»; en 1 «No le has capturado»	Correcta
15	Texto no numérico en un menú (error)	abc	Aviso y nueva petición	El programa se detiene con NumberFormatException	Fallo (limitación conocida, 6.3)

Fragmento real de una ejecución (pruebas 4, 5 y 6), de la versión inicial del programa: opción inválida, curación y curación sin poción. Tras la opción 9 se aprecia que la vida del jugador baja aunque no ha realizado ninguna acción; es el fallo ya corregido (6.1, punto 1).

```
Tu tipo es agua y el de tu rival es planta porque tiene a Terteg
```

```
Tu vida es #### 40.00 y la del rival es #### 40.00
Elecion: 9
Por favor, elige una opción válida del 1 al 7.
Tu vida es # 16.59 y la del rival es #### 40.00
Elecion: 2
Has gastado tu ultima pocion
Tu vida es ### 36.59 y la del rival es ###### 60.00
Elecion: 2
No te has podido curar porque no tienes pociones
Tu vida es # 18.97 y la del rival es ###### 60.00
```

Fragmento real del modo historia (prueba 13): tras ganar el primer combate aparece el menú del mundo y se encuentra un pekemen salvaje.

```
Elecion: 3
¡El Terteg del rival ha sido derrotado!
Has ganado
Que quieres hacer 1. luchar contra el lider o 2. luchar contra pekemens salvajes
2
Has salido: Bedef
Que quieres hacer matar(1) o capturar(2)
```

Conclusiones de las pruebas. Las acciones de combate, el menú, la condición de victoria y la de derrota funcionan como se diseñó. Las pruebas sacaron a la luz dos limitaciones reales: (1) una entrada no numérica detiene el programa porque `Integer.parseInt` lanza una excepción que no se captura, limitación que se mantiene en esta versión (6.3, punto 14); y (2) una opción inválida en combate hacía perder el turno, fallo que ya está corregido (6.1, punto 1).

4.2. Cómo se documentan las pruebas con aleatoriedad

Como el juego usa números aleatorios, cada prueba indica sus condiciones de partida (equipo, pekemen salvaje, estado activo) y lo que se comprueba es el comportamiento, no cifras concretas. Las pruebas de probabilidad, como la captura del 60 %, se repiten 100 veces y se compara el número de éxitos con el rango esperado. Cuatro intentos, como en la prueba 14, no bastan para demostrarla.

4.3. Plan de pruebas del modo historia

Continúa la numeración de la tabla del apartado 4.1 (pruebas 16 a 28) y cubre los casos que faltaban: líderes, rival final, equipo completo, captura, experiencia, cambio forzoso, estados alterados y probabilidad de captura. Las columnas «Resultado obtenido» y «Estado» se rellenan al ejecutar cada prueba sobre la versión corregida del programa.

N.º	Prueba	Condiciones y entrada	Resultado esperado	Resultado obtenido	Estado
16	Derrotar al líder de agua	Modo historia: ganar el primer combate y elegir «luchar contra el líder». Vencer a sus 3 pekemens.	Al caer el último pekemen del líder se muestra la victoria y se vuelve al menú del mundo.		Por ejecutar
17	Derrotar al líder de fuego	Igual que la prueba 16 con el líder de fuego.	Igual que la prueba 16.		Por ejecutar
18	Derrotar al líder de planta	Igual que la prueba 16 con el líder de planta.	Igual que la prueba 16.		Por ejecutar
19	Combate contra el rival final	Haber derrotado a los tres líderes y elegir «luchar contra el líder».	El rival usa un equipo de 6 pekemens. Al derrotarlos a todos termina la aventura.		Por ejecutar
20	Equipo con 6 pekemens	Capturar salvajes hasta llenar el equipo (6). Abrir el cambio de pekemen en un combate.	Se listan los pekemens con vida distintos del activo. No hay errores.		Por ejecutar
21	Captura con huecos libres	Equipo con menos de 6. Capturar un salvaje (opción 2).	La criatura se guarda en el primer hueco libre y no se produce ninguna excepción.		Por ejecutar

N.º	Prueba	Condiciones y entrada	Resultado esperado	Resultado obtenido	Estado
22	Captura con el equipo completo	Equipo con 6. Capturar un salvaje con éxito.	No se añade ninguna criatura, el programa avisa y no se cierra.		Por ejecutar
23	Experiencia y subida de nivel	Matar salvajes seguidos hasta sumar 100 de experiencia. Anotar la experiencia tras cada salvaje.	Todo el equipo recibe la experiencia. Al llegar a 100 el pekemen sube de nivel.		Por ejecutar
24	Cambio forzoso al caer el activo	Equipo con 2 o más pekemens. Dejar que caiga el activo.	Se pide elegir otro pekemen con vida y el combate continúa sin quedar bloqueado.		Por ejecutar
25	Derrota en el modo historia	Dejar caer a todos los pekemens del jugador en un combate de la historia.	Se muestra «Has perdido el combate» y se vuelve al menú principal.		Por ejecutar
26	Duración de los estados alterados	Usar el efecto especial con fuego (quemadura), planta (veneno) y agua (ahogar) contra un tipo distinto. Contar los turnos.	Quemadura −15 y veneno −25 por turno. El ahogamiento derrota al rival a los 3 turnos.		Por ejecutar
27	Estados al cambiar de pekemen	Tener un estado activo en el rival y cambiar de pekemen propio.	Se anota qué ocurre con el estado y se comprueba que coincide con lo descrito en la memoria.		Por ejecutar
28	Probabilidad de captura (60 %)	100 intentos de captura con huecos libres. Registrar cuántas tienen éxito.	Resultado cercano a 60 de 100 (se acepta entre 50 y 70, unas dos desviaciones típicas).		Por ejecutar

5. Dificultades, soluciones y ampliaciones

5.1. Dificultades encontradas y soluciones

- **Guardar y modificar los datos de cada criatura.** Al principio no estaba claro cómo representar varios datos de distinto tipo. Se resolvió con una matriz de texto en la que cada fila es una criatura, y conversiones con parseInt / parseDouble para operar.
- **Cambios que afectaban al catálogo.** Al asignar una fila directamente, los equipos compartían el mismo vector y bajar la vida de una criatura alteraba el catálogo. Se solucionó copiando con clone().
- **Equipos incompletos.** Los huecos vacíos (null) provocaban errores al recorrer el equipo; se resolvió comprobando != null antes de leer cada posición.
- **Controlar el final del combate.** Hubo que coordinar varios bucles anidados (menú, combate, turnos y mundo) con break y variables indicadoras para salir exactamente del bucle correcto.
- **Efectos con duración.** El ahogamiento necesita contar turnos; se implementó con un contador que actúa como estado (9 = inactivo, 0 a 3 = en curso).

5.2. Limitaciones conocidas

Estas limitaciones corresponden a la primera versión de la memoria; su estado actual se recoge en el apartado 6.

- Las entradas no numéricas detienen el programa (prueba 15). Se mantiene en esta versión (ver 6.3, punto 14).
- Una opción inválida de combate consumía el turno (prueba 4). Corregida (ver 6.1, punto 1).
- Se comparaban textos con == en lugar de equals. Corregido: ahora se usa `.equals()` (ver 6.1, punto 2).
- El cálculo de la ventaja de tipos está repetido varias veces en el código (ver 6.3, punto 19).
- La experiencia ganada solo se guarda en la matriz cuando el pekemen sube de nivel (ver 6.3, punto 15).

- La vida se guarda como entero al cambiar de pekemen (ver 6.3, punto 16).
- El golpe crítico del rival usa la ventaja del jugador y su ataque normal no lleva ventaja (ver 6.3, punto 17).
- Protegerse no bloquea el golpe crítico del rival (ver 6.3, punto 18).

5.3. Ampliaciones propuestas para la segunda evaluación

- Validar toda la entrada con try / catch, cuando se estudie el tratamiento de excepciones, para que un texto no numérico no cierre el programa.
- Resolver las limitaciones del apartado 6.3 (experiencia, vida decimal, ventaja del rival y protección frente al crítico).
- Agrupar el código repetido (ventaja de tipos, lectura de menús, cálculo de daño) en métodos.
- Sustituir las matrices de texto por clases (Pekemen, Equipo) con atributos tipados y usar ArrayList.
- Añadir una opción explícita para consultar el estado del equipo (vida, nivel y experiencia de cada pekemen).
- Guardar y cargar partidas en un fichero, y añadir más tipos, objetos y ataques distintos por criatura.
- Documentar y probar de forma sistemática con una tabla de casos y un registro de errores corregidos.

6. Revisión tras el feedback del 6-11-26

Tras recibir el feedback del profesor se han revisado los puntos señalados en el código. Este apartado recoge, punto por punto, qué se ha corregido, qué son decisiones de diseño razonadas y qué limitaciones se reconocen y quedan pendientes. El estado de cada punto corresponde a la versión actual de App.java.

6.1. Correcciones realizadas

N.º	Punto revisado	Explicación
1	Opción inválida en combate consume el turno	Corregido. La opción se valida dentro de un do-while antes de procesar el turno: si no está entre 1 y 7 se muestra el aviso y se vuelve a pedir la opción sin que el rival actúe. Pendiente de repetir la prueba 4 sobre esta versión.
2	Comparación de cadenas con ==	Corregido. Las comparaciones de tipos se hacen con .equals(), que compara el contenido del texto y no la referencia. Además se usan las constantes FUEGO, AGUA, PLANTA y NORMAL, también en el catálogo de pekemens, para mantener un único criterio.
3	Nomenclatura y tipo boolean	Corregido. Las variables siguen la convención camelCase y las variables de control pasan de Boolean (clase envoltorio) a boolean (tipo primitivo).
4	Pekemen inicial del rival	Corregido (detectado al revisar el código). En el switch que elige el pekemen inicial del rival faltaban los break, de modo que siempre se ejecutaba el último caso y se mostraba un aviso de error. Con los break, el rival inicial vuelve a ser aleatorio.
5	Reinicio del modo historia	Corregido. Los marcadores de progreso del modo historia se reinician al empezar cada partida, de modo que una partida nueva no hereda el estado de la anterior. Pendiente de comprobar con las pruebas 16 a 19 y 25.
6	Captura con huecos libres en el equipo	Revisado. new String[6][8] crea las seis filas del equipo; lo que vale null son sus elementos. Los huecos libres se detectan con tusPekemens[contador][NOMBRE] == null y el acceso a esa posición es válido. Pendiente de repetir las pruebas 21 y 22 (captura con huecos y con equipo completo).

6.2. Decisiones de diseño y puntos que no afectan al programa

N.º	Punto revisado	Explicación
7	Acción «Explotar»	Decisión de diseño. Acción de alto riesgo y alta recompensa: derrota al pekemen rival activo a cambio de la mitad de la vida actual del jugador. Se reconoce que es muy poderosa; su justificación, su efecto en el equilibrio y sus límites están en el apartado 1.6.
8	Curación y vida máxima	Decisión de diseño. La poción recupera siempre 20 puntos de vida y solo se puede usar una vez por combate; no se ha definido un tope de vida máxima, por lo que un pekemen puede quedar por encima de su vida inicial. Al ser un único uso, el efecto queda limitado.
9	Estados alterados	Decisión de diseño. Se procesan con if / else if, por lo que en cada turno solo se aplica el de mayor prioridad (quemadura, veneno, ahogamiento). Duración, acumulación y comportamiento al cambiar de pekemen en el apartado 1.6.
10	Derrota y victoria en el mismo turno	Decisión de diseño. Se comprueba primero la caída del pekemen del jugador: si cae, se procesa su derrota (y la del combate si no le quedan más pekemens) aunque el del rival también haya caído en ese turno.
11	Cambio obligatorio al caer un pekemen	Decisión de diseño. Cuando cae el pekemen activo y quedan otros con vida, el jugador debe elegir uno para poder continuar el combate.
12	Vida negativa y barra de vida	No afecta al funcionamiento. Tal como está diseñado el programa, un pekemen con vida igual o menor que 0 se da por derrotado y se sustituye antes de volver a dibujar la barra de vida.
13	Respuestas sí/no	Decisión de diseño. En la captura y en el cambio de pekemen, una respuesta distinta de «si» se trata como «no» (en la captura se muestra el aviso «si o no» y se pasa a la siguiente posición). Se mantiene así en esta versión.

6.3. Pendiente para la segunda evaluación

N.º	Punto revisado	Explicación
14	Entrada no numérica	Limitación conocida, no corregida. Cuando se introduce un texto en un menú, Integer.parseInt lanza una NumberFormatException y el programa se cierra (prueba 15). La solución prevista con try / catch corresponde a contenidos que aún no se han estudiado en clase, por lo que se incorporará cuando se vea el tratamiento de excepciones.
15	Experiencia cuando no se llega a subir de nivel	Pendiente. La experiencia solo se escribe en la matriz cuando el pekemen alcanza 100 puntos y sube de nivel; si no llega, la experiencia ganada en ese momento no se conserva. Se corregirá en la segunda evaluación junto con la progresión de niveles.
16	Vida al cambiar de pekemen	Pendiente. Al cambiar de pekemen la vida se guarda en la matriz como entero, por lo que se pierde la parte decimal, y la vida solo se escribe en la matriz al cambiar o al caer. Se corregirá guardando el valor double al final de cada turno.
17	Ventaja de tipos en el ataque del rival	Pendiente. El golpe crítico del rival reutiliza la ventaja calculada desde el pekemen del jugador y su ataque normal no aplica ventaja. Se corregirá calculando una ventaja propia del rival.
18	Protección frente al golpe crítico	Pendiente. Protegerte anula el ataque normal y el ataque desesperado del rival, pero no su golpe crítico. Se corregirá comprobando la protección también en esa rama.
19	Código repetido	Pendiente. El cálculo de la ventaja de tipos está repetido en cuatro puntos del código; se agrupará en un método en la segunda evaluación.

Estas correcciones no añaden funcionalidades nuevas: el código, la memoria y el repositorio de GitHub deben reflejar la misma versión. Los puntos del apartado 6.3 son limitaciones conocidas que se mantienen en esta versión y se reconocen expresamente en la memoria.